Mono2Sim-GS: Calibration-Light RGB Real-to-Sim with LingBot-Guided Gaussian Splatting

Edge-Cloud Monocular Reconstruction for Hybrid Mesh-Gaussian Assets

Student Name(s)
Computer Vision Final Project
NYU Shanghai

Abstract

Robots can collect monocular RGB video cheaply, but turning that video into a scene representation useful for simulation, navigation, and visual inspection remains difficult. A simulator-facing scene needs explicit geometry for collision, occupancy, and planning, while a camera-facing scene needs visually realistic rendering. These requirements are poorly served by a single representation: meshes and point clouds are geometrically useful but visually incomplete, while 3D Gaussian splats provide strong real-time rendering but are not directly simulation-ready. This project builds and evaluates Mono2Sim-GS, a calibration-light RGB real-to-sim reconstruction layer that combines low-latency edge tracking, streaming learned geometry, and backend feed-forward Gaussian visual reconstruction. On the edge side, a HikRobot RGB camera and Jetson-class device run RGB preview, cuVSLAM tracking, trajectory output, and asynchronous LingBot colored-point-cloud updates. On the backend GPU side, offline video or sparse multi-view keyframes are processed by LingBot and GenWildSplat to produce pose, depth, colored geometry, and Gaussian visual assets. The main methodological contribution is LingBot-Guided Gaussian Splatting (LG-GS; the Route B line in our experiment log): instead of asking GenWildSplat to infer pose, depth, scale, and appearance from weakly-overlapping video keyframes, we inject LingBot depth, intrinsics, and camera poses to initialize Gaussian means and synchronize views in a shared world frame. We evaluate the design on a 10-second, 301-frame indoor RGB video and a live HikRobot/cuVSLAM/LingBot pipeline. The results show that local GenWildSplat chunks can look sharp but fail global continuity; single global 32–44 context reconstructions increase coverage but become blurry and memory-heavy; and LG-GS improves pose synchronization and global consistency while still requiring photometric Gaussian optimization for final visual quality. The final artifact is not a complete simulator, but a working visual reconstruction layer and edge-cloud backbone toward a hybrid mesh-Gaussian real-to-sim scene bundle.

1 Introduction

RGB video is the easiest sensing modality for many robots and mobile devices. A phone, webcam, or robot camera can capture a room without special hardware, but downstream embodied systems need more than a sequence of images. They need camera motion, spatial structure, persistent scene memory, visual appearance, and eventually physical geometry that can be used by navigation stacks and simulators. This project focuses on a calibration-light setting: the system should work from an uncalibrated or weakly calibrated monocular RGB stream, rather than assuming a carefully calibrated RGB-D rig or a dense offline photo collection. This gap motivates the project question:

> *Can we turn a calibration-light monocular RGB video or robot RGB stream into a pose-aligned 3D scene layer that is useful as the first stage of a real-to-sim pipeline?*

The system is designed around two input modes. In **live edge mode**, a HikRobot RGB camera and Jetson-class edge device provide RGB preview, tracking, trajectory, and rolling colored point-cloud updates. This mode emphasizes responsiveness and latest-available geometry. In **offline backend mode**, a recorded video or sparse multi-view keyframes are sent to a backend GPU, where LingBot and GenWildSplat produce higher-quality pose/depth/point-cloud outputs and Gaussian visual assets. The same GS Console WebUI is used to inspect both modes.

The long-term real-to-sim goal is a scene bundle containing meshes, occupancy maps, navigation geometry, and photorealistic visual assets. This project focuses on the reconstruction layer before full simulator export. The current output consists of:

- a camera trajectory $T_{\text{world}\leftarrow\text{cam}}(t)$,
- dense or semi-dense depth and colored point-cloud geometry P_{colored},
- a pose-aligned 3D Gaussian visual asset G_{visual},
- a GS Console inspection interface for live RGB, point clouds, trajectory, NAV2-style projection, and RGB-vs-Gaussian view comparison.

This scope is intentionally narrower than a complete real-to-sim simulator. We do not claim completed TSDF fusion, watertight mesh extraction, navigation costmap export, or Isaac Sim integration. Instead, we study the visual and geometric reconstruction layer that such a simulator would need.

We also do not claim that a full global Gaussian map can be generated in three seconds. Instead, second-level Gaussian generation is used for local sparse-view visual windows on the backend GPU, while global continuity is maintained by LingBot geometry and future TSDF/mesh fusion.

1.1 System Positioning

Mono2Sim-GSis best understood as a bridge between robot-side perception and backend visual reconstruction. It is not a pure SLAM system, because it also produces Gaussian visual assets. It is not a pure Gaussian reconstruction system, because global pose/depth/point-cloud geometry is treated as a first-class output. It is not a complete simulator, because the physical mesh branch is still future work. Table 1 summarizes this position.

Table 1: System positioning of Mono2Sim-GS.

Layer	**Current role**	**Real-to-sim purpose**
Edge tracking	HikRobot RGB, cuVSLAM pose, live trajectory	responsive robot-side state
Learned geometry	LingBot depth, confidence, colored point cloud	scaffold for mapping, TSDF, mesh, and Gaussian alignment
Visual Gaussian	GenWildSplat and LG-GS Gaussian asset	photorealistic camera observation and RGB-vs-GS inspection
Physical geometry	planned TSDF/mesh/occupancy export	collision, navigation, and Isaac Sim integration

1.2 Why a Hybrid Representation?

A real-to-sim scene has two different jobs. The physical side must support collision checking, occupancy reasoning, local planning, and simulation geometry. For these tasks, explicit depth, TSDFs, meshes, or occupancy maps are appropriate. The visual side must produce realistic camera observations for inspection, policy evaluation, and future vision-language-action systems. For this task, neural rendering and 3D Gaussian Splatting are attractive because they render view-dependent appearance efficiently [1].

Using only one representation creates a mismatch. A mesh can be navigable but visually flat. A Gaussian scene can look realistic but may contain low-opacity floating artifacts, inconsistent scale, no watertight boundary, and no direct collision surface. We therefore use a hybrid design:

$$\text{geometry layer} \rightarrow \{\text{depth, points, TSDF, mesh, occupancy}\}, \tag{1}$$

$$\text{visual layer} \rightarrow \{\text{3D Gaussians, RGB rendering, view comparison}\}. \tag{2}$$

1.3 Core Idea

The project combines three complementary systems and assigns each to the role it is best suited for:

- **cuVSLAM** provides low-latency visual odometry suitable for edge robotics deployments [2]. In our live stack, it acts as the real-time tracking frontend.
- **LingBot-Map** provides learned streaming geometry: camera poses, depth, and colored point clouds [3]. In our stack, it acts as the dense geometry and mapping backend.
- **GenWildSplat** provides a fast feed-forward Gaussian prior from sparse, unposed images [4]. In our stack, it acts as the backend visual asset generator rather than as the only global mapper.

The live architecture is inspired by the frontend/backend separation in SLAM systems: the frontend should track quickly, while the backend can refine geometry more slowly. LingBot is a strong streaming 3D reconstruction backbone, but running a large transformer checkpoint and camera/depth heads as the only live perception path is expensive on edge hardware. Therefore, our live pipeline uses cuVSLAM for the low-latency tracking path and schedules LingBot dense point-cloud updates asynchronously. This makes LingBot usable in a near-real-time robot-side preview loop without forcing every RGB frame to produce a new dense reconstruction.

The main insight from our visual experiments is that GenWildSplat is strong as a sparse-view visual prior but is not, by itself, a robust long-video global mapper. In a 10-second video, early frames show a sofa/table region, while later frames move toward a hallway with weak overlap and low-texture walls. More keyframes do not automatically make a feed-forward sparse-view model more coherent. Instead, they can force multiple weakly-overlapping local scenes into one prediction, causing blur and misalignment.

We address this with LingBot-Guided Gaussian Splatting, abbreviated as LG-GS. The implementation was originally called Route B in the experiment log, but the more descriptive name is LingBot-Guided Gaussian Splatting because it explains the method: LingBot supplies geometry

and GenWildSplat supplies Gaussian appearance. Original GenWildSplat predicts camera pose, depth, and Gaussians from images:

$$I_{1:k} \rightarrow \{\hat{T}_i, \hat{D}_i, \hat{G}\}. \tag{3}$$

LG-GS changes the responsibility split:

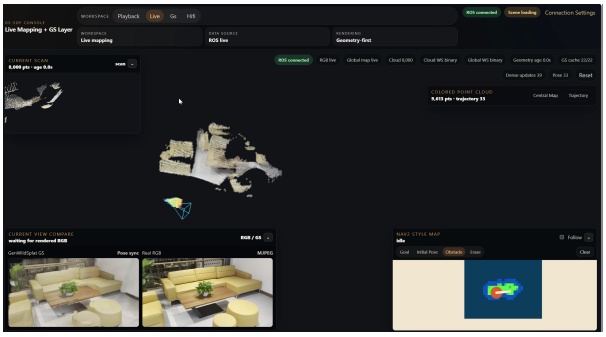
$$I_{1:k}, K_i, T_i^{\text{LingBot}}, D_i^{\text{LingBot}} \rightarrow G_{\text{pose-guided}}. \tag{4}$$

LingBot supplies the geometric scaffold. GenWildSplat supplies visual Gaussian attributes. The WebUI then uses LingBot global camera poses to synchronize real RGB and Gaussian rendering.

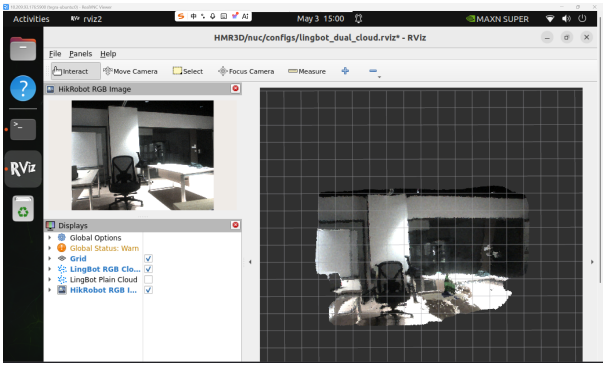
1.4 Contributions

This project makes five concrete contributions.

1. **A working calibration-light edge-cloud reconstruction backbone.** The system connects HikRobot/ROS RGB input, Jetson-side cuVSLAM pose tracking, asynchronous LingBot dense geometry, A6000-side GenWildSplat Gaussian generation, and GS Console visualization.
2. **A hybrid real-to-sim representation design.** We separate geometry for physical use from Gaussians for visual rendering, avoiding the common overclaim that a raw Gaussian asset is automatically simulation-ready.
3. **LingBot-Guided Gaussian Splatting.** We modify the reconstruction path so LingBot depth, intrinsics, and poses initialize Gaussian means in a shared world frame and stabilize RGB-vs-Gaussian view synchronization.
4. **A multi-rate systems design.** Fast RGB and pose streams remain responsive while dense LingBot mapping and Gaussian reconstruction run asynchronously or on the A6000 side.
5. **An ablation-style experimental study.** We compare raw local chunks, overlapping chunk atlases, single global multi-keyframe reconstructions, preprocessing choices, display cleanup, and LG-GS variants.



(a) Project-generated RGB-vs-Gaussian inspection screenshot.



(b) Live preview, point cloud, and reconstruction console.

Figure 1: Working artifacts from the project deck. The current system is a reconstruction and inspection layer toward real-to-sim, not a completed simulator export.

2 Related Work

2.1 Classical SfM, MVS, and COLMAP

Classical structure-from-motion (SfM) and multi-view stereo (MVS) solve the geometric version of our problem: given overlapping views of a scene, estimate camera poses and recover sparse or dense geometry. COLMAP is a representative robust offline system, combining feature extraction and matching, incremental or global pose estimation, bundle adjustment, and dense MVS reconstruction [5, 6]. The core innovation of this family is not a single neural architecture but a carefully engineered optimization pipeline: features provide repeatable correspondences, bundle adjustment enforces multi-view consistency, and MVS densifies geometry once camera poses are known.

This line of work strongly motivates our emphasis on camera gauge, scale, and cross-view consistency. It also explains why our raw GenWildSplat results fail when pose is unstable: realistic rendering cannot compensate for a broken geometric frame. However, classical SfM/MVS does not directly solve our target setting. It typically assumes sufficient overlap, textured surfaces, and offline optimization over a collection of images. It also does not directly provide a live robot-side colored point-cloud preview, asynchronous edge/backend scheduling, or a Gaussian visual layer for WebUI inspection. Our project is therefore not a replacement for COLMAP. It is a calibration-light monocular-video pipeline that uses learned geometry and learned Gaussian priors when classical overlap, texture, or calibration assumptions are weak.

2.2 TSDF and Volumetric Fusion

Volumetric fusion methods address the physical geometry side of real-to-sim. A truncated signed distance field (TSDF) integrates multiple depth observations into a single implicit surface volume. The classic method of Curless and Levoy [7] and systems such as KinectFusion [8] show how depth and pose can be fused into a stable surface representation. The key contribution is temporal integration: individual depth maps are noisy and partial, but repeated observations can produce a surface that is more useful for geometry processing.

This is exactly the kind of representation needed for collision, navigation, occupancy, and Isaac Sim geometry. It also defines a clear boundary for our project: a Gaussian visual asset should not be asked to serve as a physical mesh. TSDF fusion still needs reasonably accurate depth and poses, which is why LingBot is valuable as a monocular depth/pose provider. At the same time, TSDF or mesh outputs are usually not photorealistic and do not model view-dependent appearance as well as neural rendering. Our final target is therefore hybrid: LingBot depth/pose should feed a TSDF/mesh branch for physical use, while Gaussian reconstruction provides a visual observation branch.

2.3 NeRF and Neural Radiance Fields

Neural Radiance Fields (NeRF) introduced a powerful formulation for novel view synthesis by learning an implicit radiance field from posed images [9]. The central innovation is differentiable volume rendering: color and density are queried along camera rays, enabling the model to represent view-dependent appearance and synthesize unseen views. NeRF reframed 3D reconstruction as a rendering-supervised optimization problem rather than only a geometry-estimation problem.

This is an important conceptual ancestor of our visual branch. We also care about the camera observation that a robot or simulator would see from a new viewpoint. However, standard NeRF-style optimization is not ideal for our system goal. Per-scene optimization can be slow, interactive WebUI inspection is less direct than with explicit primitives, and collision/navigation export remains inconvenient because the representation is not naturally a mesh or occupancy map. Our project inherits the neural-rendering objective but chooses 3D Gaussian Splatting and GenWildSplat because they provide a more explicit, streamable, and viewer-friendly visual layer.

2.4 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) represents scenes with explicit anisotropic Gaussian primitives and renders them through differentiable splatting [1]. Each Gaussian has a center, scale/rotation or covariance, opacity, and appearance parameters. The key innovation is the combination of explicit point-like primitives with fast rasterization, giving high-quality real-time radiance-field rendering.

This representation is highly compatible with our GS Console and Spark/WebUI inspection goals. A Gaussian PLY or SPZ chunk can be loaded, rendered, cached, and compared against RGB frames. It is also easier to reason about Gaussian centers and coordinate frames than with a fully implicit neural field. The limitation is equally important: raw Gaussians are not physical assets. They can contain floating artifacts, opacity noise, scale ambiguity, duplicate surfaces, and holes. They do not directly define a watertight collision mesh. We therefore restrict Gaussians to the visual layer and reserve TSDF/mesh outputs for simulation geometry.

2.5 Gaussian SLAM and SplaTAM

Gaussian SLAM systems show that 3D Gaussians can be used not only for offline rendering but also for online tracking and dense mapping. SplaTAM, for example, tracks camera motion and maps a scene using 3D Gaussians from RGB-D input [10]. The key idea is to make the map itself renderable and optimizable: tracking can compare the rendered Gaussian map against incoming observations, while mapping updates Gaussian parameters.

This inspires our view that Gaussians can become an online visual memory. But our setting is different. SplaTAM and related dense Gaussian SLAM systems often assume RGB-D input or a known camera setup. Our system begins from calibration-light monocular RGB video and separates the pipeline into a fast tracking frontend, a learned dense geometry backend, and a sparse-view Gaussian visual prior. We are not directly implementing a Gaussian SLAM backend. Instead, we use LingBot to provide monocular geometry and GenWildSplat to provide a feed-forward Gaussian appearance prior.

2.6 cuVSLAM and Frontend/Backend Decoupling

cuVSLAM is a CUDA-accelerated visual SLAM system designed for edge robotics and Jetson-class hardware [2]. It explicitly separates a low-latency frontend for online pose estimation from a backend for asynchronous map refinement, loop closure, and pose graph optimization. This separation is central to our system design. A robot-side pipeline cannot wait for every dense reconstruction update before publishing pose or RGB preview.

In our implementation, cuVSLAM acts as the live tracking frontend, while LingBot provides slower learned dense geometry and colored point-cloud updates. This is not a rejection of LingBot's pose head; it is a systems decision. LingBot's transformer inference and camera/depth heads are valuable but expensive, especially on edge hardware. cuVSLAM gives the live loop a stable low-latency trajectory, and LingBot updates the geometry/mapping branch asynchronously. This makes the overall system closer to a practical robot stack: fast tracking first, dense learned mapping when compute is available.

ıon and 60 FPS (except stereo-inertial mode at 30 FPS).

Configuration	Track call time, ms		Jetson HW utilization	
	Desktop	Jetson	CPU %	GPU %
Mono	0.9	2.7	NA	NA
Mono-Depth	5.9	15.1	2.6	55.0
Stereo-Inertial	1.3	3.8	1.3	2.2
Stereo	0.4	1.8	5.5	1.7
Multicamera (2-stereo)	0.8	2.0	8.3	9.0
Multicamera (3-stereo)	1.4	2.3	8.3	11.0
Multicamera (4-stereo)	2.1	NA	NA	NA

Table 1 details CPU and GPU utilization percentages for cuVSLAM visual odometry
3.2 with RealSense D435/D455 stereo cameras. All live tests utilized cameras cc
ion and 60 FPS, except for Stereo-Inertial mode, which ran at 30 FPS with IMU
ed framerate was necessary to ensure valid IMU based integration, providing

Figure 2: cuVSLAM paper crop used to motivate the edge tracking frontend. The important connection to our project is not only the raw timing numbers, but the frontend/backend design: low-latency pose estimation is separated from slower asynchronous map refinement.

2.7 LingBot-Map as Learned Geometry Backbone

LingBot-Map is a streaming 3D reconstruction model built around a geometric context transformer [3]. Its key design is geometric context attention, which maintains an anchor context for coordinate grounding, a local pose-reference window for dense local geometry, and trajectory memory for long-range consistency. This directly addresses the main weakness of naive long-video feed-forward reconstruction: either the model forgets long-range context, or memory grows too quickly.

LingBot is particularly suitable as our geometry backbone because it predicts pose, depth, confidence, and point clouds from monocular video. The paper reports around 20 FPS at 518×378 inputs and shows strong long-sequence performance. On Oxford Spires in the sparse setting, LingBot-Map reports AUC@15 of 61.64 and ATE of 6.42, compared with DA3 at 49.84/12.87 and VGGT at 23.84/24.78; among online methods, the paper reports a large gap over CUT3R at 5.98/18.16. The dense trajectory setting is also relevant: when sequence length increases from 320 to 3,840 frames, LingBot-Map's ATE changes only from 6.42 to 7.11. These numbers support our choice to use LingBot as the geometric scaffold for Gaussian reconstruction.

However, LingBot alone does not solve the full visual real-to-sim problem. A colored point cloud is useful for geometry preview and mapping, but it is not the same as a photorealistic camera observation layer. It may be sparse, noisy, or visually incomplete. This is why our system pairs LingBot with GenWildSsplat: LingBot provides pose/depth/scale grounding; GenWildSplat provides a learned Gaussian appearance prior.

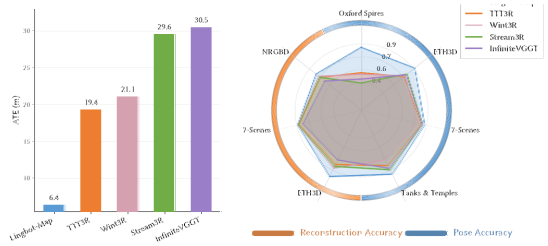


Figure 1. Comparison of LingBot-Map with State-of-the-Art streaming reconstruction methods.

(a) LingBot overview crop.

Table 3. Sparse vs. dense trajectory accuracy on Oxford Spires. We compare ATE under the sparse (320 frames) and dense (3,840 frames) settings. ΔATE measures the degradation from sparse to dense. LingBot-Map maintains near-constant accuracy while competing methods degrade significantly. The **best** and second best results are marked.

	CUT3R	TTT3R	Wint3R	Inf.-VGGT	Stream3R-w	LingBot-Map
ATE_{sparse} ↓	18.16	19.35	21.10	30.49	33.03	**6.42**
ATE_{dense} ↓	32.47 (+14.31)	25.05 (+5.70)	32.90 (+11.80)	31.75 (+1.26)	33.73 (+0.70)	**7.11** (+0.69)
FPS ↑	**29.21**	28.97	3.88	7.78	13.66	20.29

In the *dense* setting (full 3,840 frames), we stress-test long-sequence streaming capabilities (Tab. 3). This setting

(b) LingBot comparison table crop.

Figure 3: LingBot paper crops. These support the choice of LingBot as the learned streaming geometry backbone: it is designed for pose, depth, and point-cloud recovery from continuous video rather than only static sparse-view reconstruction.

2.8 GenWildSplat, AnySplat, and Sparse-View Feed-Forward Gaussians

GenWildSplat belongs to a newer family of feed-forward sparse-view Gaussian reconstruction methods. It builds on the idea of AnySplat-style prediction, where a transformer processes unposed images and predicts depth, camera parameters, and Gaussian attributes in a single pass [11, 4]. GenWildSplat extends this direction to unconstrained sparse images with appearance variation and transient objects. Its paper emphasizes 2–6 unposed input views, a single feed-forward pass, no per-scene optimization, and approximately three-second reconstruction for sparse local scenes.

This is exactly why it is attractive for our backend GPU branch. It can quickly generate local

Gaussian visual assets from sparse multi-view keyframes. But the same strength becomes a limitation for long robot video. A 301-frame trajectory is not simply a larger sparse-view input. If keyframes span weakly-overlapping regions, the model must infer a global scene, pose graph, scale, depth, and appearance at once. Our experiments confirm this: local chunks can look sharp, while a single global 32–44 context reconstruction becomes blurrier and memory-heavy.

LG-GS is our response to that mismatch. We do not use GenWildSplat as a fully pose-free long-video mapper. We reinterpret it as a pose-guided visual Gaussian prior. LingBot pose, depth, and intrinsics provide the geometric frame and stabilize Gaussian means; GenWildSplat provides appearance, opacity, scale, and other Gaussian attributes. This is the main methodological connection between our project and sparse-view feed-forward Gaussian reconstruction.

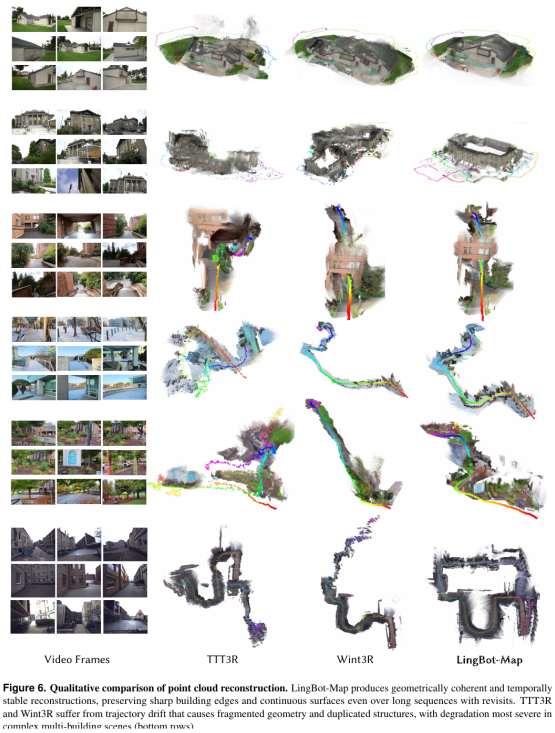


Figure 6. Qualitative comparison of point cloud reconstruction. LingBot-Map produces geometrically coherent and temporally stable reconstructions, preserving sharp building edges and continuous surfaces even over long sequences with revisits. TTT3R and Wint3R suffer from trajectory drift that causes fragmented geometry and duplicated structures, with degradation most severe in complex multi-building scenes (bottom rows).

(a) LingBot qualitative geometry crop.

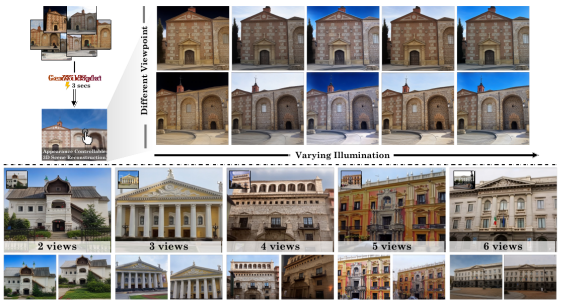


Figure 1. **GenWildSplat** reconstructs 3D scenes from sparse, unposed images with varying illumination and transient objects in a single 3-second feed-forward pass, and no per-scene optimization is required. Given 2–6 input views, our method predicts novel views under target lighting conditions while handling occlusions. **Top:** Novel-view synthesis under different lighting from the same sparse inputs, demonstrating appearance control. **Bottom:** Reconstruction quality across varying input sparsity (2–6 views), showing view-consistent rendering even with minimal observations. In each block, the top-left image (inset) is an input view; the remaining images are novel-view

(b) GenWildSplat sparse-view visual prior crop.

Figure 4: The two learned components used by our system have complementary strengths: LingBot provides streaming geometry and camera structure, while GenWildSplat provides fast sparse-view Gaussian visual reconstruction.

: 1. Comparison of key characteristics across existing meth-
nd our approach. Unlike optimization-based or feed-forward
ines, our method is fast, capable of sparse views, view-
stent, and generalizes to novel lighting conditions.

hod	In-the-Wild	Fast	View-Consis	Few-Views
mization-Based [16, 36]	✓	✗	✓	✗
l-Forward Based [11]	✗	✓	✗	✓
	✓	✓	✓	✓

(a) Poor generalization to test views

(b) Prior methods fail on sparse-views

(c) Fails under unseen or extreme lighting

:e 2. **Limitations of Prior Work.** Prior methods [16, 36]
nder sparse-view conditions. **(a) Overfitting:** Scene-specific
ıization produces artifacts and geometric spikes with small
:ra perturbations. **(b) Camera dependency:** Methods rely on
MAP for pose estimation, which fails under sparsity. Even
higher-quality transformer-based poses (e.g., VGGT), recon-
tions exhibit severe artifacts and blurring. **(c) Limited ap-
ance adaptation:** Test-time optimization cannot adapt to
l lighting, causing color bleeding and geometric distortions
ı target illumination differs from training conditions.

ing conditions. However, the absence of such multi-

(a) Sparse-view method comparison crop.

Table 4. **Ablation study** evaluated on the MegaScenes Dataset.

Model Variant	PSNR ↑	SSIM ↑	LPIPS ↓
w/o Appearance adapter	13.76	0.391	0.405
w/o Occlusion handling	15.14	0.405	0.513
w/o Curriculum learning	11.72	0.318	0.438
Full model (Ours)	15.84	0.440	0.407

Figure 9. **Ablation Study.** Removing the appearance adapter, occlusion handling, or curriculum causes major failures: fixed appearance, baked-in transient objects, or color collapse. With all components enabled, GenWildSplat produces clean, consistent 3D reconstructions.

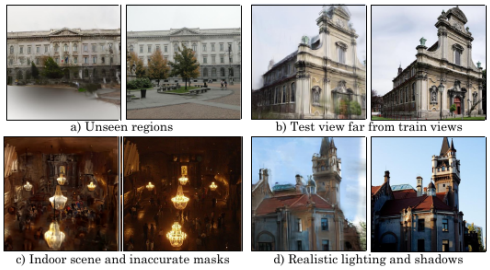


Figure 10. **Limitations.** (a) missing geometry in sparsely observed regions, (b) artifacts and double geometry for test views distant from training views, (c) degraded performance in indoor environments with imperfect occlusion masks, and (d) absence of shadow modeling and realistic relighting.

and Tab. 4. Removing the appearance adapter prevents the
model from capturing appearance variations, resulting in a

(b) GenWildSplat qualitative/limitation crop.

Figure 5: GenWildSplat paper crops. The method is a strong sparse-view feed-forward Gaussian prior, but its target setting is not identical to long calibration-light robot video. This motivates our LG-GS geometry-guided variant.

2.9 SyncFix and Diffusion-Based Refinement

SyncFix uses diffusion-based multi-view synchronization to refine rendered outputs and improve cross-view consistency [12]. This is valuable for producing visually coherent rendered images or videos after a reconstruction method has generated initial views. GenWildSplat uses SyncFix as a post-processing step for visualization in some qualitative results.

For our project, SyncFix is not a direct solution to raw asset quality. It improves 2D rendered output consistency, not the 3D structure of a PLY loaded in Spark or GS Console. It does not create collision geometry, remove all floating splats from the underlying asset, or align a long-video scene with LingBot poses. We therefore treat SyncFix as future visual refinement, while the current project focuses on making the raw Gaussian asset pose-guided and inspectable in 3D.

2.10 Real-Time Embodied Systems

The final motivation is embodied deployment. Systems such as latency-aware vision-language-action navigation highlight that robot perception, reasoning, mapping, and control often run at different rates [13]. This supports our architectural choice: fast pose and RGB preview should not block on slower dense geometry or cloud-side Gaussian generation. For a robot acting in a changing environment, a timely partial state can be more useful than a delayed complete reconstruction.

3 Data

3.1 Offline RGB Video

The main offline experiment uses a short indoor RGB video. It contains a sofa/table region in early frames and a hallway/wall region in later frames. This sequence is useful because it is simple enough to inspect visually but hard enough to expose long-video reconstruction failures: later views have weak overlap with early views, and the hallway contains low-texture walls and boundary regions.

The processed video contains:

- approximately 10 seconds of RGB video,
- 301 frames,
- LingBot input width 518,
- depth maps of approximately 294×518,
- per-frame intrinsics, extrinsics, depth confidence, and colored point outputs.

The main local artifact directory recorded during development was:

`CVPR/nuc_output/video_real2sim_playback/lingbot_vid30fps_518_full/`

with outputs such as:

`lingbot_predictions.npz`, `lingbot_summary.json`.

3.2 Live RGB Stream

The live pipeline uses a HikRobot RGB camera, ROS topics, cuVSLAM tracking, LingBot dense geometry, and GS Console visualization. The live system is not evaluated as a full-rate Gaussian reconstruction engine. Instead, it evaluates whether an edge-cloud reconstruction backbone can keep RGB preview and tracking responsive while scheduling dense geometry asynchronously.

3.3 Preprocessing Variants

Image preprocessing strongly affects both field of view and pixel-depth alignment. We tested three variants:

- **Default square crop.** This follows a square GenWildSplat-style input, but it crops away left and right image regions. In our video, this removed parts of the right wall, poster, and hallway edge.
- **Wide LingBot crop.** This preserves the 518×294 LingBot-like aspect ratio and keeps RGB, depth, and Gaussian initialization aligned.
- **Letterbox.** This preserves the full wide field of view while padding to a square input. It reduces crop loss but introduces padded regions that may shift the model distribution.

The current active LG-GS variant uses the wide 518×294 path because it best matches LingBot depth and the WebUI RGB preview.

4 Methods

4.1 System Overview

The system is organized as an edge-cloud reconstruction pipeline. Fast components stay on the edge or near the camera; heavy reconstruction can run offline or on an A6000-class GPU.

```
RGB video / live robot RGB stream
|
|-- fast path
|  -> RGB preview / MJPEG stream
|  -> cuVSLAM camera pose / trajectory
|
|-- geometry path
|  -> LingBot depth, confidence, intrinsics, extrinsics
|  -> colored point cloud and trajectory
|  -> future TSDF / mesh / occupancy
|
|-- visual Gaussian path
-> raw GenWildSplat / chunk atlas / global GS
-> LG-GS pose-guided Gaussian
-> SPZ chunks and GS Console view comparison
```

Figure 6: High-level system decomposition. The current project implements the RGB, pose, LingBot geometry, Gaussian visual, and inspection paths. TSDF/mesh/navigation export remains future work.

4.2 Two-Mode Architecture

The system has two operating modes that share the same representation and WebUI but place computation differently.

Live / edge mode. The live mode is designed for a robot-side or camera-side deployment. A HikRobot RGB camera streams frames to a Jetson-class edge computer. The edge side keeps the responsive loop small:

$$\text{RGB capture} \rightarrow \text{cuVSLAM tracking} \rightarrow \text{trajectory} \rightarrow \text{WebUI preview}. \tag{5}$$

Dense reconstruction is scheduled asynchronously:

$$\text{selected frames/window} \rightarrow \text{LingBot dense update} \rightarrow \text{rolling colored point cloud}. \tag{6}$$

This is the main systems contribution on the live side. Instead of forcing LingBot's transformer inference and camera/depth heads to be the only real-time perception path, the stack uses cuVSLAM as a fast tracking frontend and LingBot as a slower dense mapping backend. This mirrors the frontend/backend split in cuVSLAM: fast local pose first, slower map refinement later. The output is a rolling point-cloud preview and trajectory that can update even when dense geometry arrives at seconds-level intervals.

Offline / backend mode. The offline mode is designed for a recorded video or sparse multi-view keyframes. It runs the heavier visual reconstruction path on a backend GPU:

$$\text{video/keyframes} \rightarrow \text{LingBot pose/depth/points} \rightarrow LG - GS \rightarrow \text{Gaussian visual asset}. \tag{7}$$

The backend GPU is where GenWildSplat-style sparse-view Gaussian generation is appropriate. In this mode, an A6000-class GPU can run LingBot or GenWildSplat on selected keyframes, export Gaussian PLY/safetensors files, and convert them into WebUI chunks. This mode is not a 3-second global reconstruction of the full 301-frame video; it is a sparse-view or local-window visual asset generation path, stabilized by LingBot geometry.

Figure 7: Hardware used for the live side: a HikRobot RGB camera mounted on a Jetson-class edge rig. The backend Gaussian experiments were run on an A6000-class GPU.

Table 2: Hardware and compute-role split.

Component	Role	Why used
HikRobot RGB camera	monocular RGB capture	calibration-light input source for live robot stream
Jetson-class edge device	RGB preview, ROS transport, cuVSLAM tracking, async LingBot worker	keeps pose and preview responsive under limited edge compute
A6000-class backend GPU	LingBot full-video processing, GenWildSplat, LG-GS export, Gaussian conversion	handles transformer inference and feed-forward Gaussian generation
GS Console WebUI	unified inspection	shows RGB, trajectory, colored point cloud, NAV2-style projection, and RGB-vs-Gaussian comparison

4.3 Workload Split

A key systems lesson is that different outputs have different timing requirements. RGB preview should be fast because it is the operator's immediate feedback. Pose should be low-latency because robot actions depend on current camera motion. Dense geometry can lag by several seconds as long as the newest available geometry is clearly labeled and updated asynchronously. Gaussian reconstruction is heavier and is best treated as an offline/cloud-side visual asset generation path.

Formally, the system should not require:

$$\text{RGB fps} = \text{new dense geometry fps} = \text{Gaussian update fps}. \tag{8}$$

Instead, the architecture uses separate rates:

$$\text{RGB preview} \gg \text{dense geometry update} \geq \text{Gaussian asset update}. \tag{9}$$

This is important for edge robotics. A robot should not pause all perception just because a dense reconstruction worker is busy.

4.4 LingBot Geometry Front-End

LingBot provides camera intrinsics K_i, camera-to-world transforms $T_i \in \mathrm{SE}(3)$, depth maps D_i, confidence maps C_i, and colored point-cloud outputs. Given a pixel $u = (x, y)$, homogeneous pixel coordinate $\tilde{u} = [x, y, 1]^T$, and depth $D_i(u)$, we back-project to camera coordinates:

$$X_c(u) = D_i(u) K_i^{-1} \tilde{u}. \tag{10}$$

Then the point is transformed into the LingBot world frame:

$$X_w(u) = T_i X_c(u). \tag{11}$$

These world-space points are used in two ways. First, they create the colored point-cloud visualization. Second, they become the geometry prior for LG-GS.

In offline backend mode, LingBot's own pose/depth outputs are used directly for the geometry scaffold. In live edge mode, the pose path is decoupled: cuVSLAM supplies the fast tracking frontend, while LingBot supplies dense geometry updates when the worker is available. Conceptually, this replaces a monolithic "LingBot predicts everything for every frame" loop with a frontend/backend architecture: cuVSLAM handles frequent pose updates, LingBot handles slower dense point-cloud mapping, and later optimization can reconcile the two branches.

4.5 Raw GenWildSplat Route

The first visual path used GenWildSplat directly. We selected keyframes from the video, exported:

`scene/images/`, `scene/masks/`

and ran GenWildSplat inference to generate:

`gaussians.ply`, `gaussians.safetensors`, `camera_poses.json`.

The raw Gaussian PLY was converted into WebUI-friendly chunks with:

`GS_Console/scripts/preprocess_gaussian_stream.mjs`.

This route verified that GenWildSplat could produce useful local visual assets, especially in the sofa/table region. The problem was coordinate gauge. GenWildSplat predicts a scene in its own arbitrary coordinate frame; LingBot and the WebUI use a different world frame. Without alignment, the RGB stream and Gaussian render do not describe the same camera trajectory.

4.6 Sim(3) Alignment for Raw Gaussians

For raw GenWildSplat outputs, we aligned GenWildSplat camera centers or point-cloud centers to LingBot camera centers using a similarity transform:

$$\min_{s,R,t} \sum_i \|sRp_i + t - q_i\|_2^2, \tag{12}$$

where p_i are GenWildSplat camera centers, q_i are LingBot camera centers, s is scale, $R \in \mathrm{SO}(3)(3)$, and $t \in \mathbb{R}^3$. This improved local alignment, but one global Sim(3) cannot fix local drift across weak-overlap chunks. It also cannot repair incorrect topology inside the Gaussian scene.

4.7 Chunk Atlas

To preserve local visual quality, we reconstructed temporal chunks independently. The initial non-overlap chunks were:

$$[0, 100], [100, 200], [200, 300]. \tag{13}$$

We then used overlapping chunks:

$$[0, 120], [60, 180], [120, 240], [180, 300]. \tag{14}$$

Each chunk was reconstructed, aligned, converted to SPZ chunks, and registered in the WebUI manifest. The UI selected the active Gaussian variant based on playback frame and cross-faded in overlap regions. This helped inspection, but it did not solve global consistency. Cross-fading hides a hard visual switch; it does not jointly optimize two Gaussian scenes into one coherent asset.

4.8 LG-GS: Pose-Guided GenWildSplat

LG-GS is the central method contribution. Instead of using GenWildSplat as a fully pose-free system, we use LingBot geometry to initialize Gaussian means. In the implementation, GenWildSplat was extended with external LingBot arguments:

- `external_lingbot_predictions_npz`,
- `external_lingbot_summary_json`,
- `external_lingbot_sample_stride`,
- `external_pts_all`,
- `external_depth_map`,
- `external_depth_conf`.

For selected keyframes, LingBot depth and camera geometry define Gaussian centers:

$$\mu_{i,u}^{\mathrm{LGGS}} = T_i^{\mathrm{LingBot}} \left(D_i^{\mathrm{LingBot}}(u) K_i^{-1} \tilde{u} \right). \tag{15}$$

GenWildSplat still predicts appearance-related attributes such as color, opacity, scale, and rotation. The responsibility split is:

$$\text{LingBot: pose, depth, scale, geometric frame,} \tag{16}$$

$$\text{GenWildSplat: visual Gaussian attributes.} \tag{17}$$

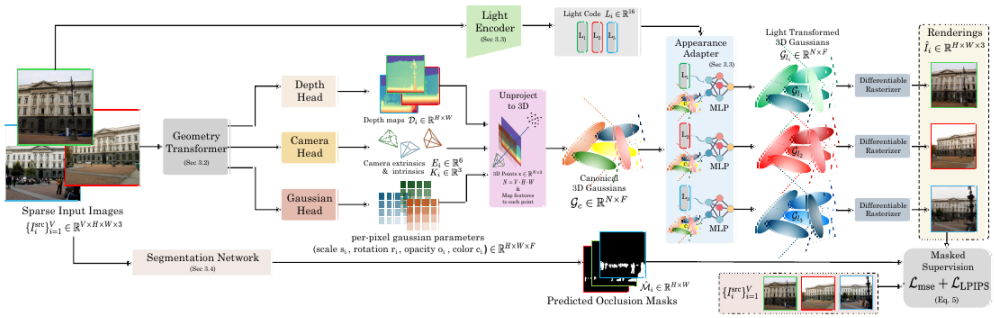


Figure 3. **Overview of GenWildSplat.** Given sparse, unposed images $\{I_i\}_{i=1}^{V}$ with appearance variations and transient objects, a geometry transformer extracts multi-view features $\mathbf{F}_i$ encoding semantic and geometric information. Specialized prediction heads process these features to output per-pixel depth $\mathbf{D}_i$, camera parameters $(\mathbf{K}_i, \mathbf{E}_i)$, and Gaussian attributes, which are unprojected into canonical 3D Gaussians $\mathbf{G}_c$. A light encoder $\mathcal{E}_{Light}$ extracts per-image lighting codes $\mathbf{L}_i = \mathcal{E}_{Light}(I_i)$. An MLP F_{light} modulates the canonical Gaussian colors using these codes: $\mathbf{G}_{\ell_i} = F_{\text{light}}(\mathbf{G}_c, \mathbf{L}_i)$. Each set of transformed Gaussians $\mathbf{G}_{\ell_i}$ is rasterized to reconstruct $\hat{I}_i$. A pre-trained segmentation network provides occlusion masks M_i to identify transient objects. Masked reconstruction loss focuses supervision on static content, enabling photorealistic, view-consistent reconstruction from sparse in-the-wild imagery.

Figure 8: Original GenWildSplat pipeline figure crop used in the presentation. LG-GS changes the grounding step: LingBot depth, intrinsics, and poses initialize Gaussian means before visual attributes are exported.

4.9 Algorithmic Summary

Input: RGB frames $I_{1:T}$, selected keyframes $\mathcal{K}$, LingBot predictions $\{K_i, T_i, D_i, C_i\}$.
Output: Pose-guided Gaussian visual asset G_{LGGS}.

1. Run LingBot on the RGB sequence to obtain depth, confidence, intrinsics, extrinsics, and colored points.
2. Select $k = 12$ keyframes spanning the 301-frame video.
3. For each keyframe and sampled pixel, back-project using Eq. 15.
4. Inject the resulting world-space points as Gaussian means in the GenWildSplat export path.
5. Export Gaussian attributes, convert PLY to SPZ chunks, and register the variant in the WebUI manifest.
6. During playback, use LingBot global camera pose for the Gaussian render camera, so RGB and GS views share the same trajectory.

Figure 9: Practical LG-GS procedure.

4.10 WebUI Inspection Console

The GS Console UI was extended from a live monitor into a reconstruction inspection console. It includes:

- a central colored point cloud and trajectory view,
- a current scan or rolling local cloud panel,
- an RGB-vs-Gaussian comparison panel,

- a NAV2-style 2D projection/debug map,
- reset playback control,
- Gaussian variant selection and chunk caching.

The transport path was also changed. RGB frames use an MJPEG fast path instead of large JSON messages. Current and global point clouds use binary WebSockets. Gaussian PLY files are converted to SPZ chunks and cached. These changes matter because early UI bottlenecks could look like reconstruction failures even when the underlying issue was data transport.

4.11 Display Calibration and Cleanup

Raw Gaussian assets frequently contain many low-opacity splats and very small scale axes. In a black-background renderer, this can appear as black speckles, grid-like holes, or overly sparse walls. We tested display-level processing:

- scale inflation,
- minimum linear scale clamping,
- opacity gamma adjustment,
- dropping very low opacity splats,
- dropping dark low-opacity splats.

These operations improve readability, but they are not true reconstruction. Increasing scale fills holes but blurs edges. Dropping splats removes noise but may remove valid surfaces. The correct next step is photometric Gaussian optimization initialized by LG-GS.

4.12 Planned Photometric Optimization

Given a LG-GS Gaussian initialization G, LingBot poses, intrinsics, and RGB frames, a local photometric optimization stage can refine visual quality:

$$\mathcal{L} = \lambda_1 \|R(G, K_i, T_i) - I_i\|_1 + \lambda_{\text{ssim}}(1 - \text{SSIM}(R_i, I_i)) + \lambda_s \mathcal{L}_{\text{scale}} + \lambda_o \mathcal{L}_{\text{opacity}} + \lambda_\Delta \|\Delta\mu\|_2^2. \quad (18)$$

Here $R(G, K_i, T_i)$ is the rendered view. The last term keeps optimized Gaussian means close to the LingBot-guided initialization. We expect this stage to be local/windowed rather than one global optimization over the entire long video.

5 Experiments

5.1 Evaluation Questions

The experiments answer five questions:

1. Can the live edge pipeline stream RGB, pose, and dense geometry without blocking?
2. Does LingBot provide usable full-video geometry for point-cloud visualization and Gaussian guidance?
3. Do raw GenWildSplat chunks work as local visual assets?
4. Does increasing the number of global keyframes solve long-video reconstruction?
5. Does LG-GS improve pose synchronization and global consistency compared with raw pose-free Gaussians?

The evaluation is mostly qualitative and systems-oriented because the project goal is reconstruction integration rather than a benchmark model. We record point counts, latency, chunk statistics, keyframe settings, splat counts, alignment errors, memory limits, and visual failure modes observed in the WebUI.

5.2 Experiment 1: Live Edge Pipeline

Table 3 summarizes the live HikRobot/cuVSLAM/LingBot/GS Console validation run. The important result is not full-rate dense reconstruction. The important result is that RGB and pose can remain responsive while dense geometry updates asynchronously.

Table 3: Live edge pipeline observations. RGB rate and new dense geometry rate are separate quantities.

Metric	**Observed value**
HikRobot RGB rate	4–5 fps
ROS RGB topic rate	4–5 fps
Rolling map point count	49,753 points
Dense updates	7
Processed dense windows	7
Trajectory length	103 poses
Saved RGB keyframes	73
Worker failures	0
Queue drops	0
Queue wait mean / max	0.011 s / 0.038 s
Worker end-to-end mean / median	5.14 s / 4.09 s
Geometry age mean / median	5.59 s / 4.47 s
Model forward mean / median	4.11 s / 3.11 s
First dense window model forward	approx. 11.54 s
Later dense model forward	mostly 1.9–3.3 s

This supports the edge-cloud design. The first dense window has a warmup cost, but later windows are faster. More importantly, no worker failures or queue drops were observed in this run. The system therefore behaves as a latest-available geometry pipeline rather than a synchronized full-rate mapping pipeline.

This result is important because it shows that LingBot can be integrated into a near-real-time Jetson-side workflow when it is not forced to own the entire live loop. The cuVSLAM frontend publishes low-latency pose and trajectory, while LingBot contributes dense learned geometry as an asynchronous mapping update. In later versions, this split can support a stronger backend: cuVSLAM tracking can provide stable motion priors, LingBot point clouds can provide dense geometry, and a mapping/refinement layer can optimize the fused scene without blocking the robot-side preview.

5.3 Experiment 2: Full-Video LingBot Geometry

The offline 301-frame video was processed with LingBot at a 518-wide input setting. The result produced depth, confidence, intrinsics, extrinsics, and colored point outputs. This experiment corrected an early failure mode: a synthetic or low-resolution depth fallback made the WebUI point cloud look wrong even though the UI pipeline was functioning. Using real LingBot depth and poses produced a much more meaningful point cloud and trajectory.

Table 4: Offline LingBot video geometry setup.

Item	**Value**
Video length	approx. 10 seconds
Frames	301
LingBot input setting	518-wide baseline
Depth resolution	approx. 294×518
Outputs	depth, confidence, intrinsics, extrinsics, colored points
Use in project	geometry visualization and LG-GS prior

5.4 Experiment 3: Raw Local GenWildSplat Chunks

The first GenWildSplat attempts used a small number of context views. The first sofa/table chunk looked visually strongest, confirming that GenWildSplat is useful as a learned visual prior. However, during full playback the Gaussian view could remain visually tied to the early sofa/table region while the RGB stream moved into the hallway. This exposed two issues:

- sparse local reconstruction does not automatically cover the whole video,
- GenWildSplat's predicted gauge does not automatically match LingBot's world frame.

5.5 Experiment 4: Overlap Chunk Atlas

We then built an overlapping chunk atlas. Table 5 shows the recorded statistics.

Table 5: Overlap chunk atlas. Lower alignment error is better. The late hallway chunk is much harder to align.

Variant	Splats	Align. mean	Observation
chunk000_120	1,039,068	0.0505	best sofa/table local quality
chunk060_180	878,408	0.0563	acceptable transition, weaker than first chunk
chunk120_240	589,799	0.0889	weaker hallway and transition geometry
chunk180_300	836,546	0.2229	visibly worse alignment and late-video quality

The chunk atlas made the best local results accessible in the WebUI, but it did not create a smooth global Gaussian scene. Independent chunks had different scale, opacity, pose, and local geometry. Cross-fade reduced hard switching but could not eliminate duplicate surfaces or inconsistent view-dependent appearance.

5.6 Experiment 5: Single Global Keyframe Count

We then tried to reconstruct the full 10-second video as one global Gaussian with more context views:

- `genwildsplat_global_32ctx`,
- `genwildsplat_global_40ctx`,
- `genwildsplat_global_44ctx_sharp`,
- `genwildsplat_global_44ctx_postopt_s05_300`,
- a 48-context attempt that exceeded the current memory budget.

The result was counterintuitive but consistent: more keyframes improved coverage but did not guarantee better quality. When context views span weakly-overlapping regions, the model must explain multiple local scenes in one feed-forward pass. The output became more complete but blurrier, and global post-optimization could not fully repair already-wrong pose/topology. This is a major lesson of the project:

> *More frames are not the same as better geometry for a feed-forward sparse-view Gaussian model.*

5.7 Experiment 6: LG-GS Variants

Table 6 summarizes the main LG-GS variants.

Table 6: LG-GS variants. The current active version is the wide stride-1 version because it best matches LingBot RGB/depth field of view and has the highest point density.

Variant	Splats	Preprocessing	Outcome
LG1 stride2	602,112	early LG-GS, stride 2	Too sparse; visible grid/gap artifacts.
LG2 wide stride2	456,876	wide 518×294, stride 2	Better FOV but still sparse; scale $4\times$ made render blurry.
LG3 letterbox	1,365,504	518×294 letterbox to square	Preserved FOV and improved density, but still had dark splats and grid artifacts.
LG4 clean	1,135,118 kept	letterbox + cleanup	Reduced black gaps by dropping low-opacity/dark splats; display-only cleanup could over-smooth.
LG5 wide stride1	1,827,504	wide 518×294, stride 1	Current active version. Best FOV consistency; right wall/poster/hallway edge less cropped; still needs photometric optimization.

The active variant uses 12 keyframes:

$$\{0, 27, 55, 82, 109, 136, 164, 191, 218, 245, 273, 300\}. \tag{19}$$

It exports approximately 1.83M splats, a 119 MB PLY, a 600 MB safetensors file, and 21 WebUI chunks totaling roughly 20 MB after chunk conversion.

5.8 Experiment 7: Field of View

The preprocessing experiment was important because a crop error can look like a pose error. With the default square crop, the RGB window may show the right wall or hallway edge while the Gaussian window lacks those regions. This mismatch makes the views appear unsynchronized even when the camera pose branch is correct.

The wide 518×294 route keeps the RGB preview, LingBot depth, and Gaussian initialization in the same aspect ratio. The letterbox route preserves full FOV while satisfying a square input, but padding may affect appearance prediction. For the current report, the wide path is the most reliable because its pixel-depth mapping is simpler and its FOV matches LingBot.

5.9 Experiment 8: Black Speckles, Grid Artifacts, and Blur

LG-GS improved synchronization but did not solve all visual artifacts. In the letterbox stride-1 variant, more than one million splats had opacity below 0.3. Combined with very small scale axes and black background rendering, this caused black speckles and grid-like wall gaps.

The cleanup variant dropped:

- opacity below 0.04,
- dark low-opacity splats with luminance below 0.16 and opacity below 0.18.

This reduced black gaps, but the trade-off remained:

- small Gaussian scale gives sharper details but more holes,
- large Gaussian scale fills holes but causes blur,
- aggressive opacity filtering removes noise but can remove valid surfaces.

The conclusion is that display calibration is useful for inspection, but final asset quality requires photometric optimization.

Figure 10: Project-generated demo strip used in the presentation. It summarizes the reconstruction and inspection workflow: RGB input, point-cloud state, Gaussian visualization, and UI-level comparison.

6 Discussion

6.1 Claim Boundary and Metric Interpretation

Several numbers in this project are easy to misread, so we make the claim boundary explicit.

The 4–5 fps number belongs to the live RGB/tracking/preview side. It describes HikRobot RGB acquisition, ROS RGB publishing, and the responsive edge-side loop. It does not mean that a new LingBot dense reconstruction or Gaussian asset is produced at 4–5 fps. Dense LingBot updates are asynchronous and seconds-level in the current live validation.

The three-second GenWildSplat claim is a sparse local visual generation claim, not a full-video mapping claim. GenWildSplat is designed for sparse unposed views such as 2–6 image inputs and fast feed-forward local reconstruction. Our 301-frame video contains long-range motion and weakly-overlapping regions. Treating the entire video as one global sparse-view problem is exactly what produced blur and memory pressure in the 32/44/48-context experiments.

LG-GS is a visual prior, not a final optimized asset. It improves camera-gauge consistency by using LingBot depth, intrinsics, and poses to initialize Gaussian means and synchronize the WebUI camera. It does not remove the need for local photometric Gaussian optimization, opacity/scale regularization, or better keyframe selection.

Physical simulation geometry still requires TSDF/mesh. The Gaussian layer is useful for photorealistic observation and visual inspection. Collision, navigation, occupancy, and Isaac Sim integration should be handled by a TSDF/mesh branch driven by LingBot depth and pose. The current project demonstrates the reconstruction backbone and visual layer; the full physical real-to-sim export remains future work.

6.2 What LG-GS Actually Fixes

LG-GS primarily fixes a geometric synchronization problem. In the raw route, GenWildSplat owns the camera gauge; in the LingBot route, the WebUI and geometry own the camera gauge. By injecting Gaussian means from LingBot depth and poses, the Gaussian scene is placed into the same world frame used by the RGB playback and point cloud. This makes RGB-vs-Gaussian comparison meaningful.

LG-GS does not automatically guarantee sharp visual quality. If LingBot depth is wrong on a wall, the Gaussian mean is also wrong. If GenWildSplat predicts unsuitable opacity or scale, the render still has artifacts. Therefore, LG-GS should be viewed as a stronger initialization, not the final optimization.

6.3 Why Local Chunks Look Better Than Global Reconstructions

Local chunks have stronger overlap and simpler geometry. The model sees a smaller, more coherent scene, so the feed-forward prior produces sharper local visual structure. Global reconstruction must cover sofa/table and hallway regions together, sometimes with weak or no direct overlap. This increases ambiguity in pose, scale, and appearance. The model then averages or stretches the scene, producing blur.

This result suggests a windowed strategy:

- use LingBot/TSDF/mesh for global continuity,
- use local optimized Gaussians for high-quality current-view rendering,
- switch or blend local visual assets using geometry-aware alignment rather than raw temporal chunks.

6.4 Why Real-Time Design Matters

The live pipeline shows that real-time behavior is a scheduling problem as much as a model problem. A robot operator or controller needs current RGB and pose quickly. Dense geometry can update at a slower rate. Gaussian reconstruction can be even slower if it is used as a visual asset generation path.

This is consistent with broader embodied AI systems: reasoning, reconstruction, mapping, and control often run asynchronously. A system that blocks fast control on slow reconstruction may look cleaner offline but behave worse online. Our design therefore keeps the fast path alive and treats dense geometry as latest-available state.

6.5 Downstream Applications

The current reconstruction layer can support several future tasks:

- **Semantic navigation**: attach object labels or language references to persistent 3D scene regions.
- **Mobile manipulation**: use geometry for obstacle avoidance and arm-base coordination.

- **Object search**: remember where objects were seen across a trajectory.
- **VLA grounding**: connect natural-language instructions to 3D visual context.
- **Digital twins**: bootstrap indoor visual assets from ordinary RGB video.
- **World-model training**: provide persistent visual scene memory for long-horizon reasoning.

These are not completed systems in this project. They motivate why the reconstruction layer matters.

7 Limitations

The current system has several important limitations.

1. **No completed physical simulator export.** TSDF fusion, mesh cleanup, collision assets, NAV2 maps, and Isaac Sim export remain future work.
2. **Monocular depth uncertainty.** LingBot depth can be inaccurate on low-texture walls, reflective boundaries, and repeated structures. LG-GS inherits those errors.
3. **Hard geometry injection.** The current LG-GS implementation directly replaces or initializes Gaussian means with LingBot-unprojected points. A softer learned conditioning mechanism may be better.
4. **Limited quantitative image metrics.** We mainly use visual inspection, point counts, latency, alignment errors, and artifact analysis. Future work should compute PSNR/SSIM/LPIPS on held-out frames after rendering with synchronized poses.
5. **Keyframe selection is still simple.** The current 12-keyframe set spans time uniformly. Motion-aware or coverage-aware selection should improve results.
6. **Display cleanup is not reconstruction.** Opacity and scale filtering improve WebUI readability but cannot replace photometric Gaussian optimization.

8 Conclusion

This project built a working RGB-first reconstruction layer toward real-to-sim scene generation. The system connects monocular RGB input, low-latency pose, LingBot learned geometry, GenWildSplat Gaussian visual priors, and a GS Console inspection UI. The main method, LG-GS, changes GenWildSplat from a pose-free sparse-view reconstruction component into a pose-guided visual prior by injecting LingBot depth, intrinsics, and camera poses into Gaussian mean initialization.

The experiments show that raw GenWildSplat is useful locally but unreliable as a long-video global mapper. Overlapping chunks preserve local sharpness but introduce switching and alignment artifacts. Single global multi-keyframe reconstructions improve coverage but often become blurry and memory-heavy. LG-GS improves pose synchronization and is the best starting point for a global visual reconstruction layer, but it still needs local photometric Gaussian optimization to reach final visual quality.

The most important design conclusion is:

> *For robot RGB video, Gaussian reconstruction should be geometry-guided and system-aware, not only image-driven.*

In the next stage, the project should fuse LingBot depth and pose into TSDF/mesh/occupancy assets, optimize LG-GS Gaussians photometrically in local windows, and export a complete scene bundle containing RGB, poses, depth, confidence, point cloud, mesh, Gaussian, navigation, and simulator metadata.

Acknowledgments

This project builds on cuVSLAM, LingBot-Map, GenWildSplat, SyncFix, SplaTAM, NeRF, 3D Gaussian Splatting, and classical reconstruction literature. Project-generated figures and screenshots are tracked in `deck/figure_sources.md`.

References

[1] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 139:1–139:14, 2023.

[2] A. Korovko, D. Slepichev, A. Efitorov, A. Dzhumamuratova, V. Kuznetsov, J. Biswas, H. Rabeti, and S. Pouya, "cuVSLAM: CUDA accelerated visual odometry and mapping," *arXiv preprint arXiv:2506.04359*, 2025.

[3] L.-Z. Chen, J. Gao, Y. Chen, N. Xue, X. Zhu, K. L. Cheng, Y. Sun, Y. Yao, Y. Xu, Y. Shen, and L. Hu, "Geometric context transformer for streaming 3d reconstruction," *arXiv preprint arXiv:2604.14141*, 2026.

[4] V. Gupta, C.-H. Lin, S. Wang, A. Bhattad, and J.-B. Huang, "Generalizable sparse-view 3d reconstruction from unconstrained images," *arXiv preprint arXiv:2604.28193*, 2026.

[5] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4104–4113.

[6] J. L. Schonberger, E. Zheng, J.-M. Frahm, and M. Pollefeys, "Pixelwise view selection for unstructured multi-view stereo," in *European Conference on Computer Vision*, 2016, pp. 501–518.

[7] B. Curless and M. Levoy, "A volumetric method for building complex models from range images," in *Proceedings of SIGGRAPH*, 1996, pp. 303–312.

[8] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, "KinectFusion: Real-time dense surface mapping and tracking," in *IEEE International Symposium on Mixed and Augmented Reality*, 2011, pp. 127–136.

[9] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *European Conference on Computer Vision*, 2020, pp. 405–421.

[10] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "SplaTAM: Splat, track and map 3d gaussians for dense rgb-d slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 357–21 366.

[11] Y. Chen, C. Zheng, H. Xu, B. Zhuang, Y. Jin, X. Xu, M. Yu, J. Pang, and F. Zhao, "AnySplat: Feed-forward 3d gaussian splatting from unconstrained views," in *ACM SIGGRAPH Asia*, 2025.

[12] D. Li, A. Yadav, C. Peng, R. Chellappa, and A. Bhattad, "SyncFix: Fixing 3d reconstructions via multi-view synchronization," *arXiv preprint arXiv:2604.11797*, 2026.

[13] Z. Huang, Y. Zhang, J. Liu, R. Song, C. Tang, and J. Ma, "TIC-VLA: A think-in-control vision-language-action model for robot navigation in dynamic environments," *arXiv preprint arXiv:2602.02459*, 2026.

A Guideline Coverage

The final project guideline asks for a mini-paper with abstract, introduction, related work, data, methods, experiments, and conclusion. Table 7 maps this report to those requirements.

Table 7: Coverage of the final report guideline.

Guideline item	**Where addressed**
Abstract	Summary of problem, approach, results, and limitations.
Introduction	Motivation, scope, hybrid representation, core idea, contributions.
Related Work	Classical reconstruction, 3DGS, Gaussian SLAM, LingBot, GenWildSplat, real-time embodied systems.
Data	Offline 301-frame video, live RGB stream, preprocessing variants, LingBot outputs.
Methods	Edge-cloud system, LingBot geometry, raw GenWildSplat, Sim(3), chunks, LG-GS, WebUI, display cleanup.
Experiments	Live metrics, full-video LingBot, chunks, global keyframes, LG-GS variants, FOV, artifact analysis.
Conclusion	Key findings, limitations, and future scene-bundle direction.

B Implementation Artifacts

Important local files and directories recorded during development:

- `CV_pre/GENWILDSPLAT_EXPERIMENT_LOG.md`
- `CV_pre/readme.md`
- `deck/speaker_notes.md`
- `deck/figure_sources.md`

- `scripts/real2sim/run_video_real2sim_playback_webui.py`
- `scripts/real2sim/inflate_gaussian_ply.py`
- `scripts/real2sim/postopt_genwildsplat_gaussian.py`
- `CVPR/third_party_research/GenWildSplat/src/eval_nvs_video.py`
- `CVPR/third_party_research/GenWildSplat/src/model/encoder/anysplat.py`
- `GS_Console/examples/web-ui/src/App.tsx`
- `GS_Console/scripts/preprocess_gaussian_stream.mjs`

C Current Active Variant

The active WebUI visual route recorded in the experiment notes is:

`genwildsplat_routeb_depth_12ctx_wide_stride1_s2`

The active WebUI URL recorded during development was:

active WebUI URL with scene and live contract parameters

D Future Scene Bundle Layout

A complete real-to-sim export should eventually use a manifest-centered layout:

```
scene_bundle/
manifest.json
rgb/
poses/
depth/
confidence/
pointcloud/
tsdf/
mesh/
gaussian_seed/
gaussian_optimized/
nav/
isaac/
reports/
```

Figure 11: Proposed final scene bundle structure.